PPL 2021.02.08

Ex 1
SCHEME:
Write a function 'depth-encode' that takes in input a list possibly containing
other lists at multiple nesting levels, and returns it as a flat list where
each element is paired with its nesting level in the original list.

E.g. (depth-encode '(1 (2 3) 4 (((5) 6 (7)) 8) 9 (((10)))))
returns
((0 . 1) (1 . 2) (1 . 3) (0 . 4) (3 . 5) (2 . 6) (3 . 7) (1 . 8) (0 . 9) (3 . 10))

Ex 2
HASKELL:
A multi-valued map (Multimap) is a data structure that associates keys of
a type k to zero or more values of type v. A Multimap can be represented as
a list of 'Multinodes', as defined below. Each multinode contains a unique key
and a non-empty list of values associated to it.

```
data Multinode k v = Multinode { key :: k
                               , values :: [v]
                               }

data Multimap k v = Multimap [Multinode k v]
```

1) Implement the following functions that manipulate a Multimap:

```
insert :: Eq k => k -> v -> Multimap k v -> Multimap k v
```
insert key val m returns a new Multimap identical to m, except val is added to the values associated to k.

```
lookup :: Eq k => k -> Multimap k v -> [v]
```
lookup key m returns the list of values associated to key in m

```
remove :: Eq v => v -> Multimap k v -> Multimap k v
```
remove val m returns a new Multimap identical to m, but without all values equal to val

2) Make Multimap k an instance of Functor.

Ex 3
ERLANG:
Consider the apply operation (i.e.<*>) in Haskell's Applicative class.
Define a parallel <*> for Erlang's lists.

Solutions

Ex 1

```
(define (depth-encode ls)
  (define (enc-aux l)
    (cond ((null? l) l)
          ((list? (car l))
           (append (map (λ (nx) (cons (+ (car nx) 1) (cdr nx)))
                        (enc-aux (car l)))
                   (enc-aux (cdr l))))
           (else (cons (cons 0 (car l)) (enc-aux (cdr l))))))
  (enc-aux ls))
```

Ex 2

```
empty :: Multimap k v
empty = Multimap []

insert :: Eq k => k -> v -> Multimap k v -> Multimap k v
insert key val (Multimap []) = Multimap [Multinode key [val]]
insert key val (Multimap (m@(Multinode nk nvals):ms))
  | nk == key = Multimap ((Multinode nk (val:nvals)):ms)
  | otherwise = let Multimap p = insert key val (Multimap ms)
                in Multimap (m:p)

lookup :: Eq k => k -> Multimap k v -> [v]
lookup _ (Multimap []) = []
lookup key (Multimap ((Multinode nk nvals):ms))
  | nk == key = nvals
  | otherwise = lookup key (Multimap ms)

remove :: Eq v => v -> Multimap k v -> Multimap k v
remove val (Multimap ms) = Multimap $ foldr mapfilter [] ms
  where mapfilter (Multinode nk nvals) rest =
          let filtered = filter (/= val) nvals
          in if null filtered
             then rest
             else (Multinode nk filtered):rest

instance Functor (Multimap k) where
  fmap f (Multimap m) = Multimap (fmap (mapNode f) m) where
    mapNode f (Multinode k v) = Multinode k (fmap f v)
```

Ex 3

```
runit(Proc, F, X) ->
    Proc ! {self(), F(X)}.
pmap(F, L) ->
    W = lists:map(fun(X) ->
```

```
                        spawn(?MODULE, runit, [self(), F, X])
                end, L),
    lists:map(fun (P) ->
                    receive
                        {P, V} -> V
                    end
            end, W).

pappl(FL, L) ->
    lists:foldl(fun (X,Y) -> Y ++ X end, [], pmap(fun(F) -> pmap(F,
L) end, FL)).
```